INTELLIGENT HOSTEL RESOURCE ALLOCATION AND RESIDENT MANAGEMENT SYSTEM

Submitted By:
Rajvardhan Singh Gahlaut

Registration Number:
23BEC0081

Institution:
VIT Vellore

PROJECT OUTCOME

The Intelligent Hostel Resource Allocation and Resident Management System was successfully developed using Java Spring Boot and MongoDB. The primary objective of the project was to automate hostel administration activities and eliminate the dependency on manual record keeping. Traditional hostel management often involves maintaining multiple registers for students, rooms, hostel cards, wardens, and occupancy records. This project provides a centralized digital platform that simplifies these operations through REST APIs and database management.

The developed system enables administrators to create and manage multiple hostels, register wardens, allocate rooms, maintain student information, and generate hostel cards. The integration of MongoDB allows flexible storage and retrieval of data using document-based collections. Swagger UI was integrated into the application to provide a user-friendly interface for testing and documenting all APIs.

The project demonstrates practical implementation of backend development concepts including CRUD operations, RESTful API design, database integration, validation mechanisms, and layered software architecture. The final system successfully stores, retrieves, updates, and manages hostel-related information while maintaining consistency and scalability. The project serves as a foundation for future enhancements such as authentication, fee management, attendance monitoring, visitor tracking, and web-based dashboards.

TOOLS USED WITH VERSION

Software and Technologies Used

Tool / Technology	Version	Purpose
Java	21	Core programming language
Spring Boot	3.3.0	Backend application framework
MongoDB	8.x	NoSQL Database
MongoDB Compass	Latest	Database visualization and management
Apache Maven	3.9.16	Dependency management and project build
Swagger OpenAPI	2.5.0	API documentation and testing
Visual Studio Code	Latest	Development environment
GitHub	Latest	Source code version control
Windows 11	Latest	Operating System

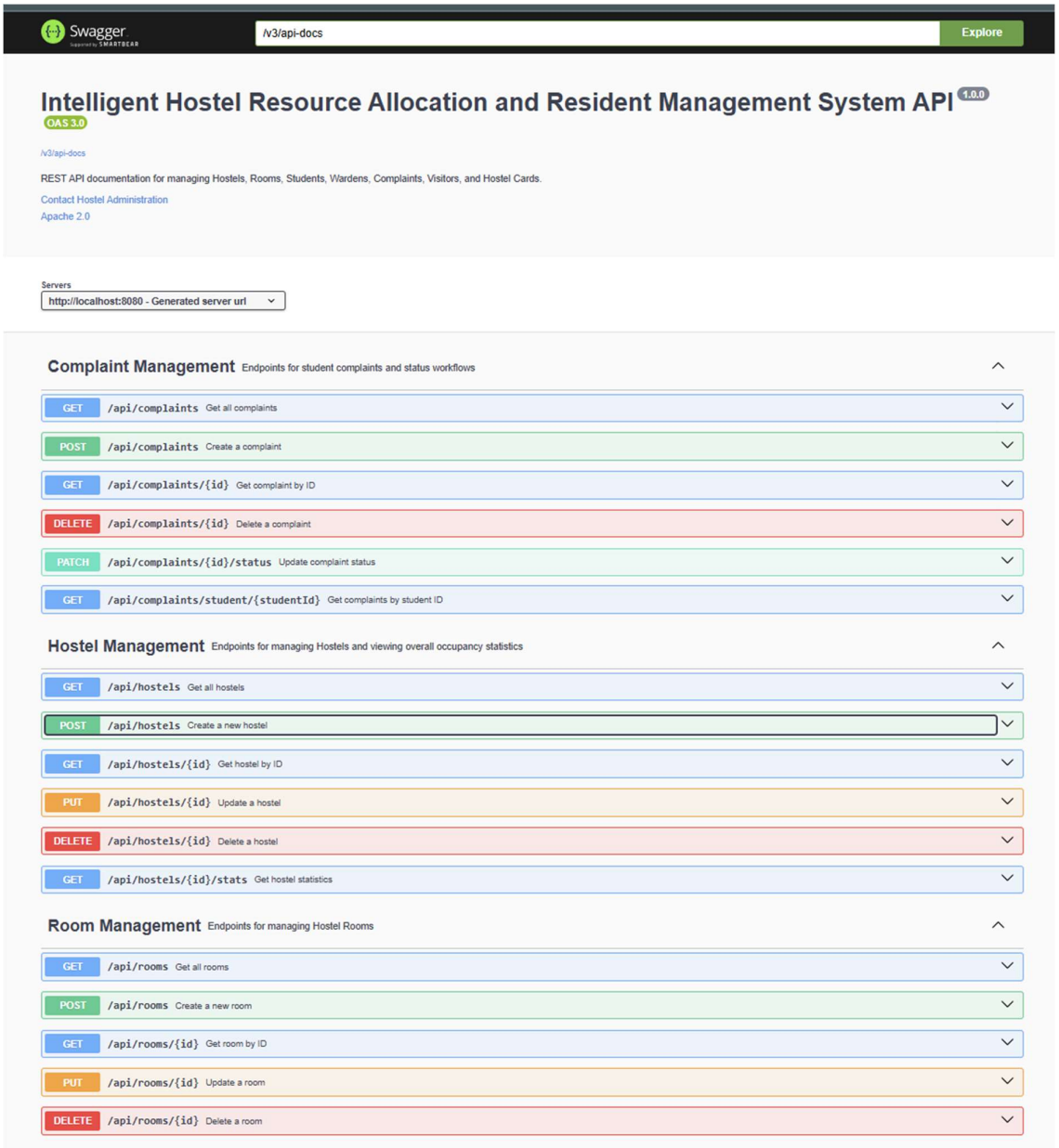
Description of Tools

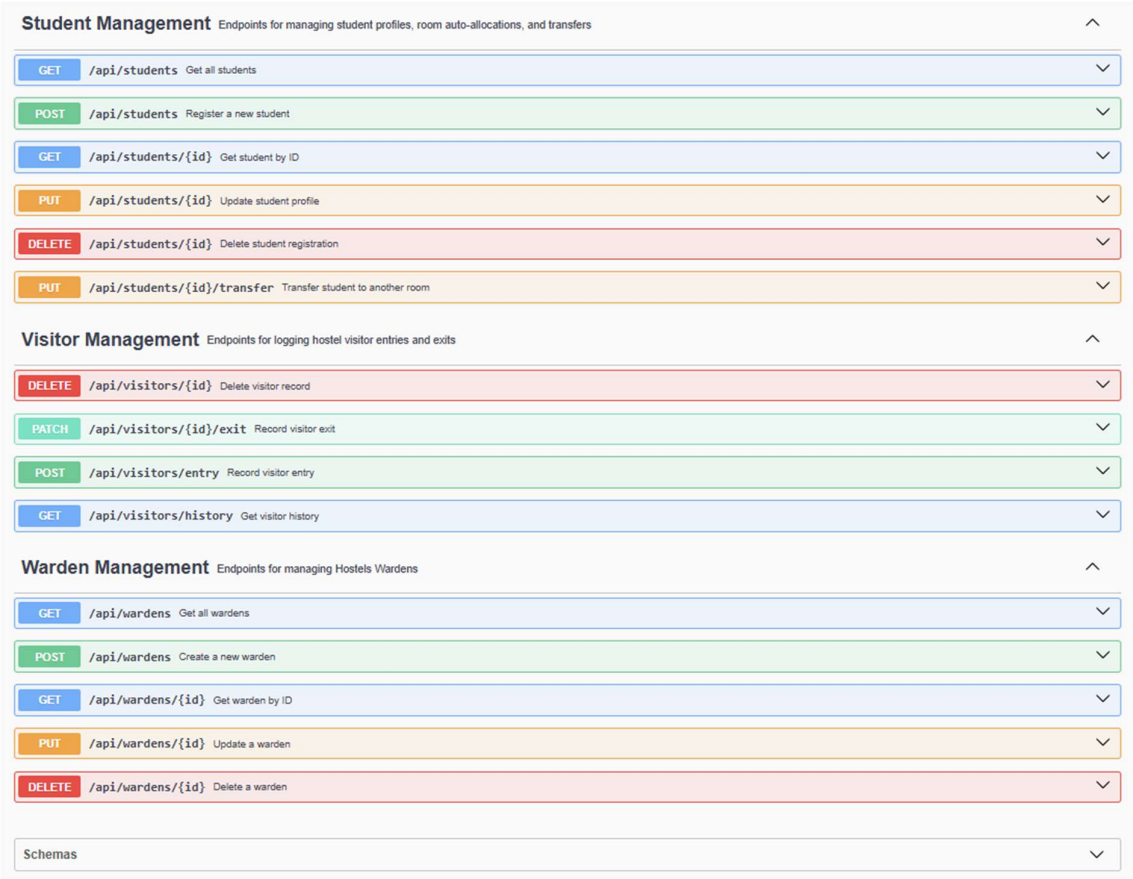
Java 21 was used as the primary programming language for implementing the application logic and backend services. Spring Boot provided the framework required for creating REST APIs and managing dependencies efficiently. MongoDB was selected as the database because of its document-oriented structure and flexibility in handling hostel-related data.

MongoDB Compass was utilized for viewing collections and verifying the stored records. Apache Maven simplified project dependency management and build processes. Swagger OpenAPI enabled interactive

testing and documentation of APIs without requiring external tools such as Postman. Visual Studio Code served as the primary development environment, while GitHub was used to maintain source code and project files.

SWAGGER UI



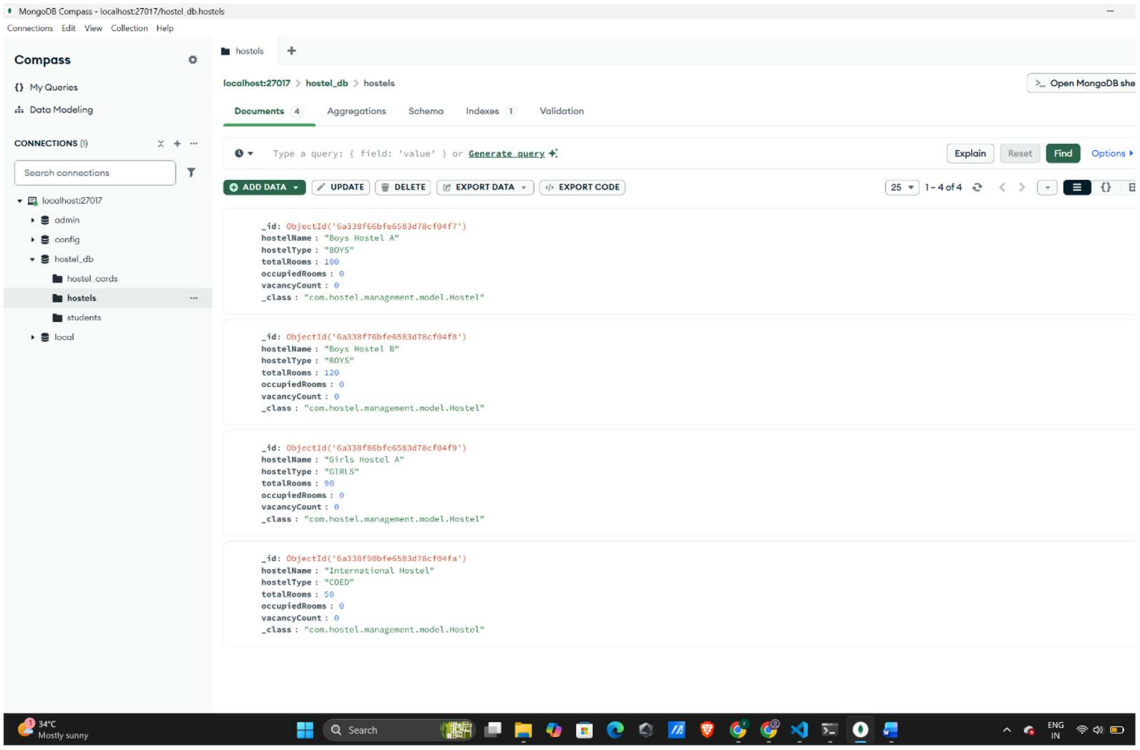


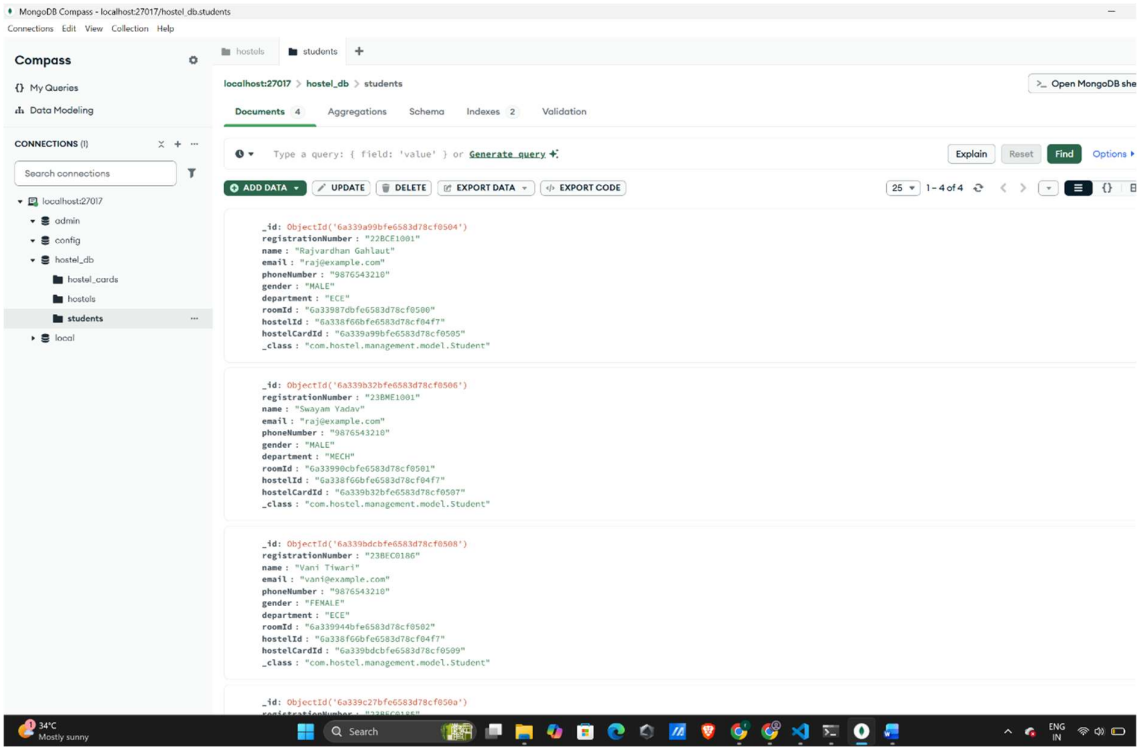
The above figure shows the Swagger UI interface generated for the Intelligent Hostel Resource Allocation and Resident Management System. Swagger acts as an API documentation and testing platform that automatically displays all available endpoints in the application.

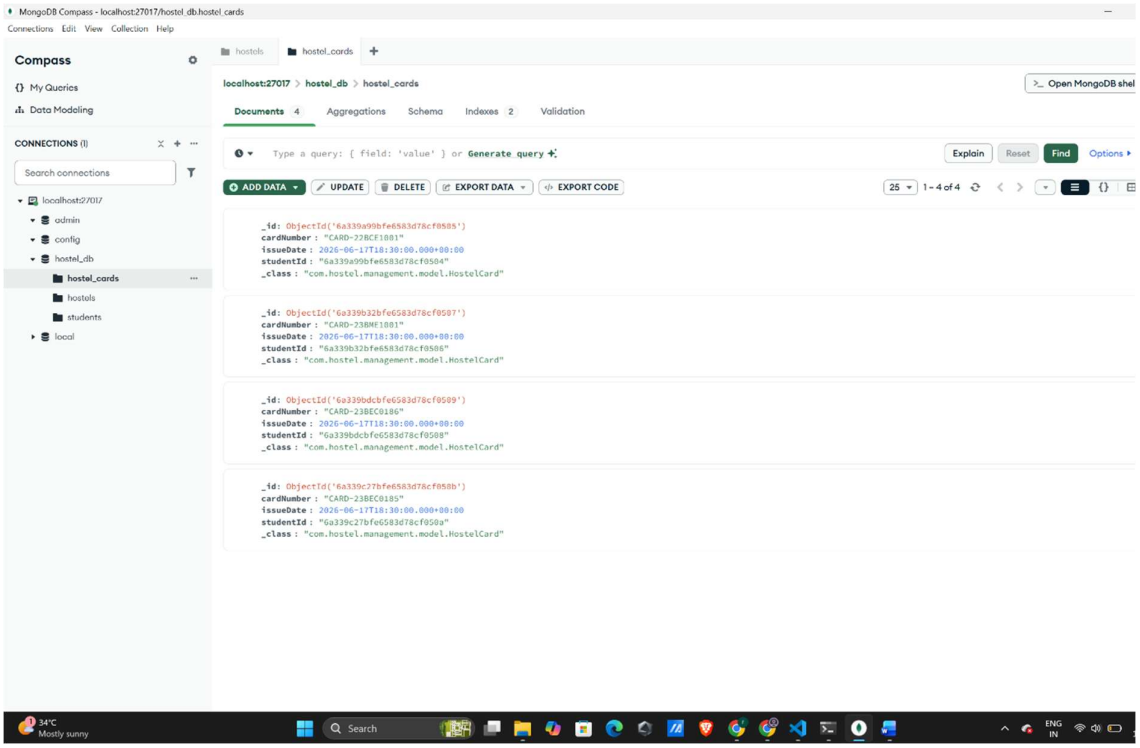
The system provides APIs for managing hostels, rooms, students, wardens, complaints, visitors, and hostel cards. Through Swagger UI, administrators can perform Create, Read, Update, and Delete (CRUD) operations directly from the browser. The interface also displays request formats, response formats, status codes, and endpoint descriptions.

The successful display of all APIs confirms that the Spring Boot application is running correctly and all controllers are properly mapped. Swagger significantly simplifies API testing and helps developers verify application functionality during development and deployment stages.

MONGODB DATABASE







Code	Details

200

Response body


```
[
  {
    "roomId": "6a33987dbfe6583d78cf0500",
    "roomNumber": "A101",
    "capacity": 4,
    "currentOccupancy": 0,
    "roomType": "BOYS",
    "hostelId": "6a338f66bfe6583d78cf04f7",
    "wardenId": "6a3394dbbfe6583d78cf04fc"
  },
  {
    "roomId": "6a33990cbfe6583d78cf0501",
    "roomNumber": "A102",
    "capacity": 4,
    "currentOccupancy": 0,
    "roomType": "BOYS",
    "hostelId": "6a338f66bfe6583d78cf04f7",
    "wardenId": "6a3395c8bfe6583d78cf04fd"
  },
  {
    "roomId": "6a33944bbfe6583d78cf0502",
    "roomNumber": "A103",
    "capacity": 4,
    "currentOccupancy": 0,
    "roomType": "BOYS",
    "hostelId": "6a338f66bfe6583d78cf04f7",
    "wardenId": "6a3395d7bfe6583d78cf04fe"
  },
  {
    "roomId": "6a33997cbfe6583d78cf0503"
```


Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu,18 Jun 2026 07:09:03 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

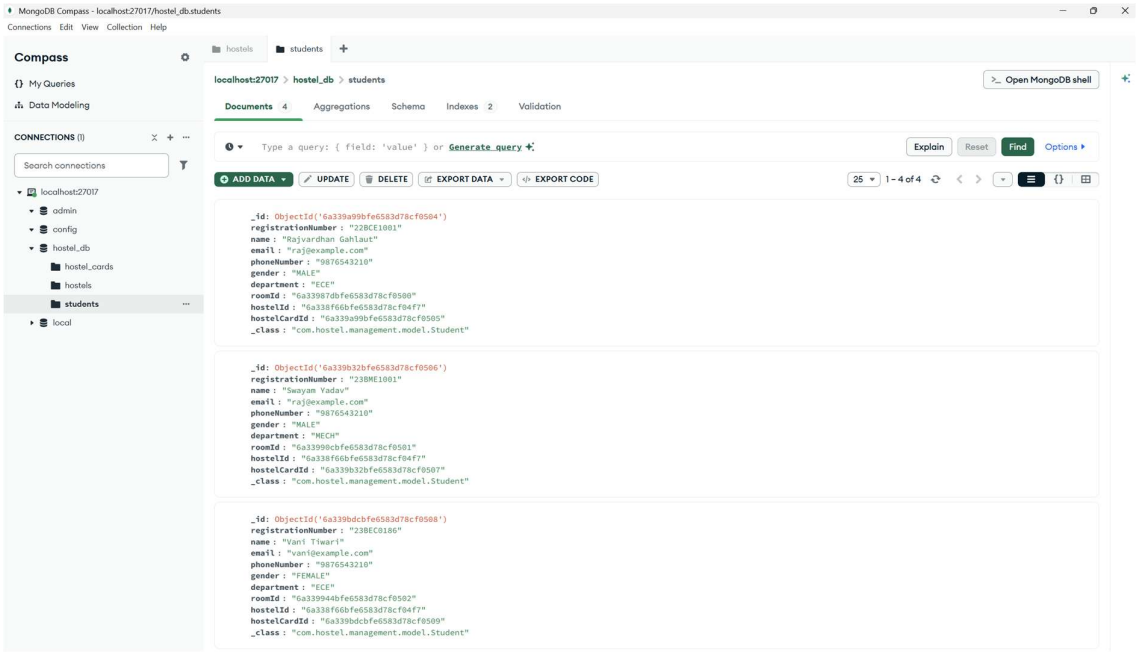
Code	Description	Links
200	OK	*No links*

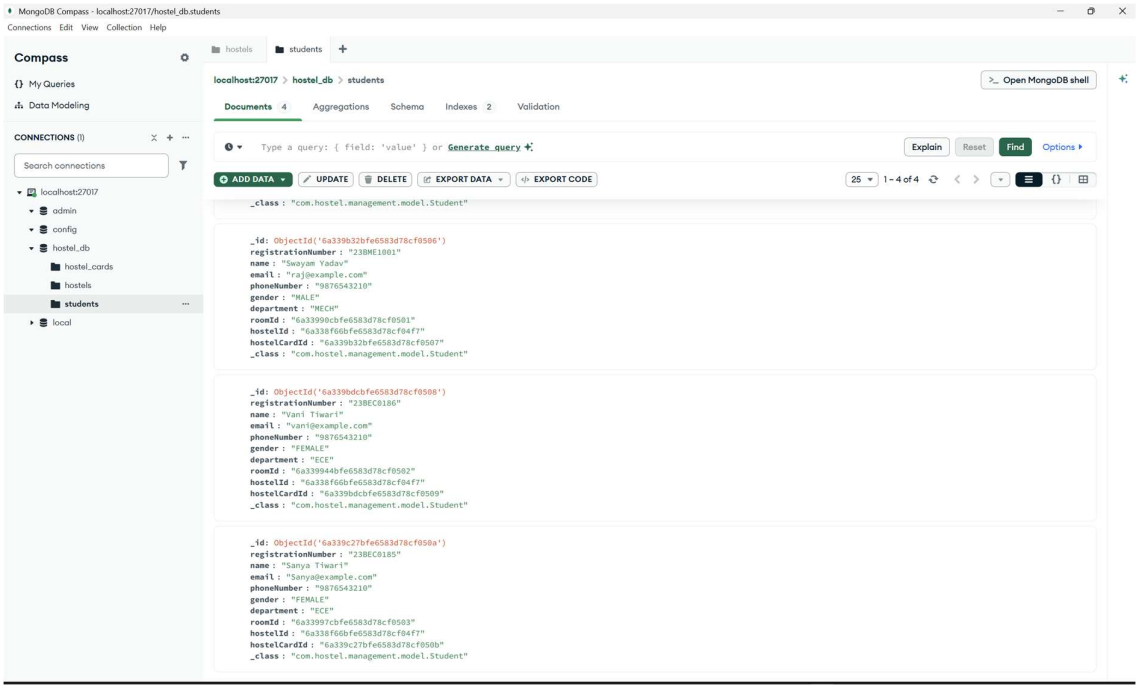
The above screenshot illustrates the MongoDB database used in the project. The database contains multiple collections including hostels, students, rooms, wardens, and hostel_cards. Each collection stores related information in document format using BSON (Binary JSON).

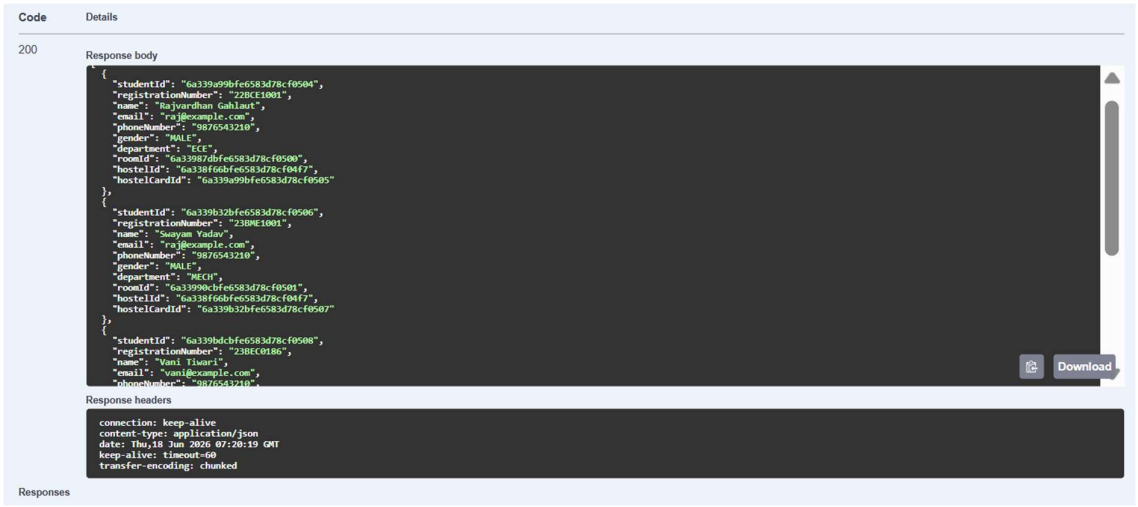
The hostels collection stores details such as hostel names, types, capacities, and occupancy information. The rooms collection contains room numbers, capacities, and hostel associations. Student records are maintained in the students collection, while hostel_cards stores hostel identification details generated for registered students.

The successful creation of these collections demonstrates proper integration between Spring Boot and MongoDB. Data entered through Swagger APIs is automatically stored in MongoDB and can be retrieved whenever required. This confirms successful implementation of persistent storage functionality.

STUDENT REGISTRATION AND ALLOCATION







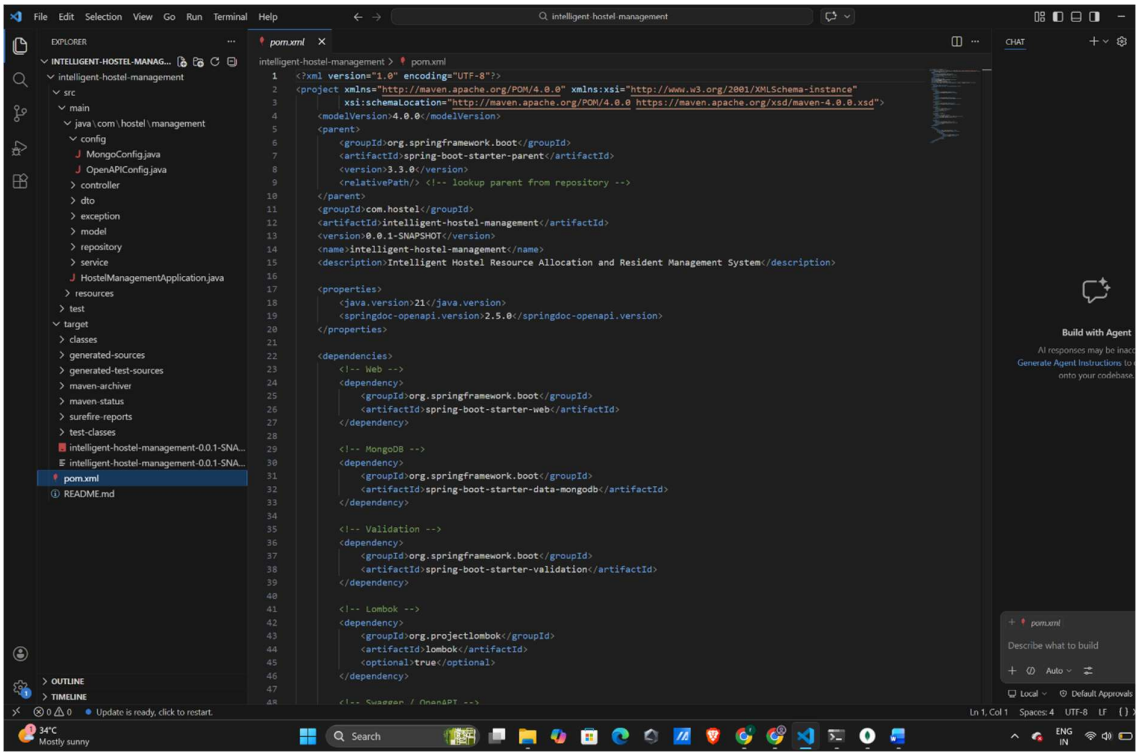
The above screenshot demonstrates successful student registration using the Student Management API. The administrator enters student information including registration number, name, email address, phone number, department, and hostel preferences.

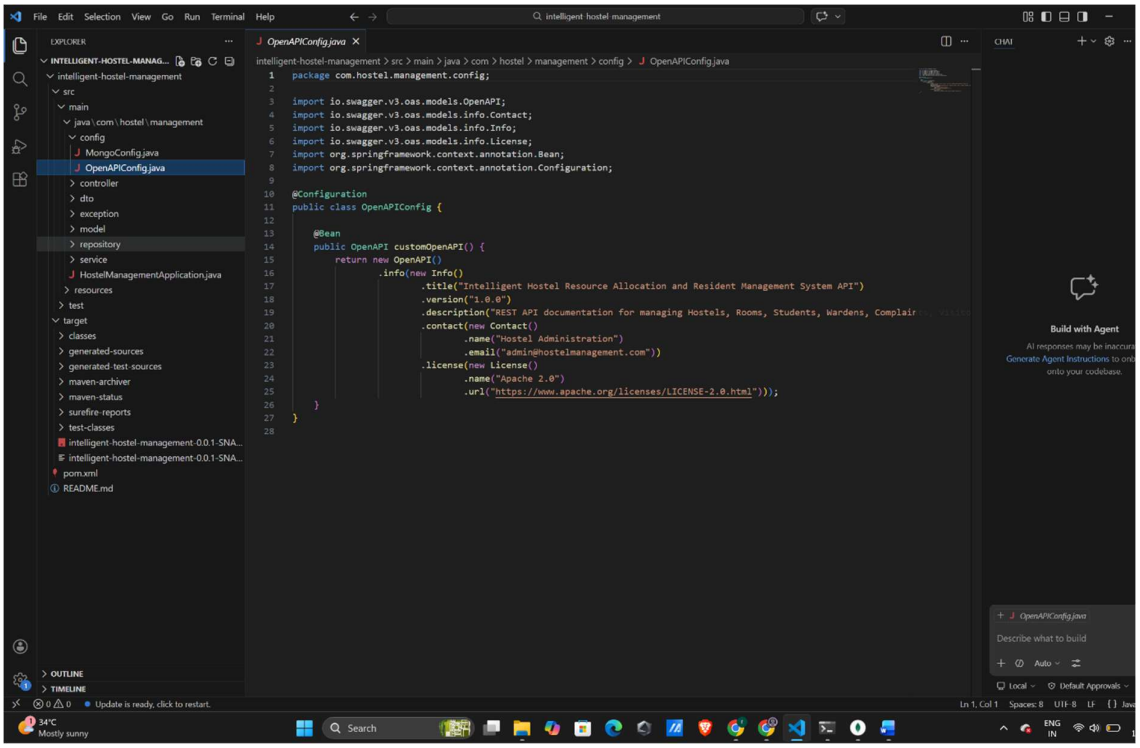
After registration, the system automatically associates the student with a hostel and room based on availability. Simultaneously, a hostel card record is generated and stored in the database. This automation reduces manual intervention and minimizes allocation errors.

The successful execution of the Student API confirms the correct functioning of business logic, database connectivity, validation mechanisms, and record management. It also demonstrates how multiple

collections interact together to maintain hostel administration data efficiently.

VISUAL STUDIO CODE PROJECT STRUCTURE






```java
package com.hostel.management.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.info.License;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenAPIConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("Intelligent Hostel Resource Allocation and Resident Management System API")
                        .version("1.0.0")
                        .description("REST API documentation for managing Hostels, Rooms, Students, Wardens, Complair
                        .contact(new Contact()
                                .name("Hostel Administration")
                                .email("admin@hostelmanagement.com"))
                        .license(new License()
                                .name("Apache 2.0")
                                .url("https://www.apache.org/licenses/LICENSE-2.0.html")));
    }
}
```

The above figure shows the project implementation in Visual Studio Code. The application follows a layered architecture commonly used in enterprise-level Spring Boot applications.

The Controller layer handles incoming HTTP requests and API endpoints. The Service layer contains business logic and processing rules. The Repository layer is responsible for database operations and communication with MongoDB. Model classes define the structure of entities such as Hostels, Students, Rooms, and Wardens. Configuration files manage MongoDB connectivity and Swagger integration.

This organized architecture improves maintainability, scalability, and code readability. The use of Maven further simplifies dependency management and project building.

SUMMARY

The Intelligent Hostel Resource Allocation and Resident Management System is a backend-based hostel management solution developed using Spring Boot and MongoDB. The system digitizes hostel administration activities by providing APIs for hostel management, room allocation, student registration, warden management, and hostel card generation.

The project demonstrates practical implementation of RESTful services, NoSQL databases, API documentation, and software architecture principles. MongoDB provides flexible data storage while Swagger simplifies API testing and verification. The developed system successfully achieves its objective of reducing manual hostel management efforts through automation and centralized data management.

CONCLUSION

The Intelligent Hostel Resource Allocation and Resident Management System was successfully designed, implemented, and tested. The application provides a reliable platform for managing hostel resources and resident information through REST APIs. Integration with MongoDB ensures efficient storage and retrieval of data, while Swagger provides a convenient environment for API testing and documentation.

The project demonstrates real-world application of backend development technologies including Java, Spring Boot, MongoDB, Maven, and Swagger. The successful creation and management of hostels, rooms, wardens, students, and hostel cards validate the effectiveness of the developed system.

Future enhancements may include user authentication, role-based access control, fee management modules, attendance tracking, visitor management, notification systems, and a web-based dashboard. These additions can transform the project into a complete hostel management solution suitable for large educational institutions.